

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030437

Name : Bandla Venkata Rama Vinay

SEC-8A

PRACTICAL-4:

Aim: Develop a C program to demonstrate the creation of multiple child processes using **fork()** and demonstrate process synchronization using **wait()** and **waitpid()**.

Observe the order in which child processes complete and understand the difference between collecting any terminated child using **wait()** and collecting a specific child using **waitpid()**.

Procedure :

1. Create the C source file **practical-4.c** containing three child-process creation operations using **fork()**.
2. Compile the program using GCC and generate the executable **Practical_4**.
3. Execute the program using **./Practical_4**.
4. Observe the PID values of the parent and three child processes.
5. Observe the behavior of **wait()** when the first child terminates.
6. Observe how **waitpid()** collects the specified child processes.

Output :

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing$ ./Practical_4
Parent PID = 1381
Using wait():
Child 2: PID = 1383
Child 1: PID = 1382
Child 3: PID = 1384
Child 2 completed
wait() collected child PID = 1383
Using waitpid():
Child 3 completed
Child 1 completed
waitpid() collected Child 1 PID = 1382
waitpid() collected Child 2 PID = 1383
waitpid() collected Child 3 PID = 1384
All children completed.
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing$ |
```

The submitted execution shows Parent PID 1381 and three child processes with PIDs 1382, 1383, and 1384. Child 2 completes first, and **wait()** collects PID 1383. The subsequent **waitpid()** calls collect Child 1, Child 2, and Child 3 by their specific PIDs. The program finally reports that all children have completed.

Code:

```
practical-4.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  #include <stdlib.h>
5
6  int main()
7  {
8      pid_t c1, c2, c3;
9      int status;
10
11     c1 = fork();
12
13     if (c1 == 0)
14     {
15         printf("Child 1: PID = %d\n", getpid());
16         sleep(3);
17         printf("Child 1 completed\n");
18         exit(1);
19     }
20
21     c2 = fork();
22
23     if (c2 == 0)
24     {
25         printf("Child 2: PID = %d\n", getpid());
26         sleep(1);
27         printf("Child 2 completed\n");
28         exit(2);
29     }
30
31     c3 = fork();
32
33     if (c3 == 0)
34     {
35         printf("Child 3: PID = %d\n", getpid());
36         sleep(2);
37         printf("Child 3 completed\n");
```

Code (continued):

```
38     exit(3);
39 }
40
41 printf("\nParent PID = %d\n", getpid());
42
43 printf("\nUsing wait():\n");
44 pid_t finished = wait(&status);
45 printf("wait() collected child PID = %d\n", finished);
46
47 printf("\nUsing waitpid():\n");
48
49 waitpid(c1, &status, 0);
50 printf("waitpid() collected Child 1 PID = %d\n", c1);
51
52 waitpid(c2, &status, 0);
53 printf("waitpid() collected Child 2 PID = %d\n", c2);
54
55 waitpid(c3, &status, 0);
56 printf("waitpid() collected Child 3 PID = %d\n", c3);
57
58 printf("\nAll children completed.\n");
59
60 return 0;
61 }
```

System Calls

System Call / Function	What it does
fork()	Creates a new child process from the calling process.
getpid()	Returns the process ID of the calling process.
wait()	Waits for any child process to terminate and collects its termination status.
waitpid()	Waits for a specified child process and collects its termination status.
sleep()	Pauses a child process for the specified number of seconds.
exit()	Terminates a child process with an exit status.

Kernel services involved

1. Process Management

fork() creates the three child processes, while **exit()** terminates each child after its work is complete.

2. Process Synchronization

wait() allows the parent to wait for a child to terminate. In the submitted execution, Child 2 completes first and its PID 1383 is collected by **wait()**.

3. Selective Child Management

waitpid() is then used with c1, c2, and c3 to collect the specified children individually.

Process flow

Parent → fork() → Child 1 + Child 2 + Child 3 → wait() → waitpid(c1) → waitpid(c2) → waitpid(c3) →
All children completed

Observation :

- The parent process has PID **1381**.
- Three child processes are created with PIDs **1382**, **1383**, and **1384**.
- Child 2 completes first because its code uses **sleep(1)**.
- Child 1 uses **sleep(3)** and Child 3 uses **sleep(2)**.
- **wait()** collects the first child that terminates, which is Child 2 with PID **1383**.
- **waitpid()** subsequently collects Child 1, Child 2, and Child 3 using their specific PIDs.
- The program displays **All children completed.** after all three children have been collected.

Result :

The C program successfully demonstrated the creation of three child processes using **fork()**. It demonstrated how **wait()** collects the first terminated child and how **waitpid()** can be used to wait for and collect specific child processes. The execution completed successfully with all child processes collected.

Practical-4 Completed Successfully